



Name:	Lyka Sophia S. Jardeleza	Date:	06/13/2025
Program:	BSCpE	Section:	A361

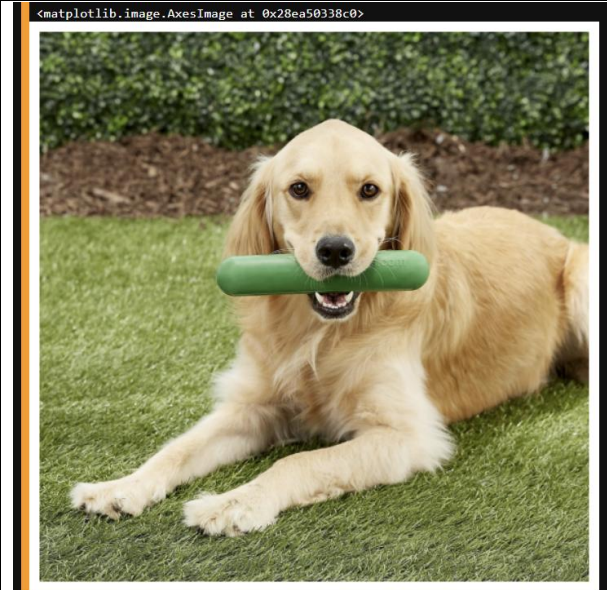
A. Guided Experiments:

(Please include the following: 1. Images or Videos of test outputs from the Python notebook and 2. Photos of the codes that was inputted in the Python notebook)

Experiment 1: Read and Display Images Using OpenCV

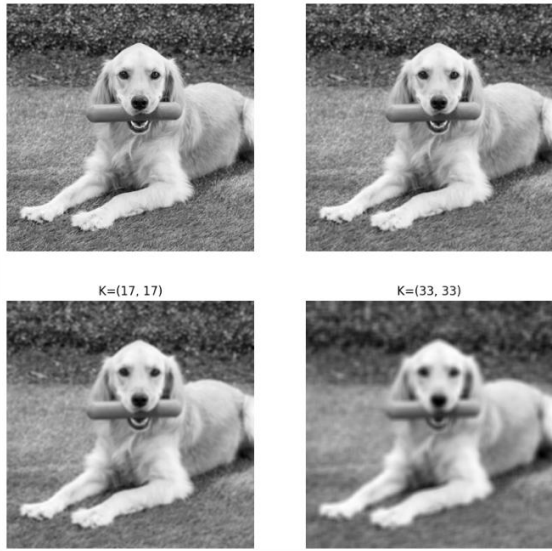
1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># READ AND DISPLAY IMAGES USING OPENCV import cv2 import wget url_image = wget.download('https://image.chewy.c om/is/image/catalog/105549_PT2._AC_S L1500_V1545242865_.jpg') read_img = cv2.imread(url_image) resize_img = cv2.resize(read_img, dsiz=(int(0.4*read_img.shape[0]), int(0.4*read_img.shape[1]))) cv2.imshow("Sample Image", resize_img) key = cv2.waitKey(5000) #Display for 5 seconds cv2.destroyAllWindows() #Release the display window</pre>

Experiment 2: Read Images Using OpenCV and Display Using Matplotlib

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># READ IMAGES USING OPENCV AND DISPLAY USING MATPLOTLIB import cv2 import wget import matplotlib.pyplot as plt url_image = wget.download('https://image.chewy.co m/is/image/catalog/105549_PT2._AC_SL1 500_V1545242865_.jpg') read_img = cv2.imread(url_image) # Convert to RGB because OpenCV default is BGR read_img = cv2.cvtColor(read_img, cv2.COLOR_BGR2RGB) plt.figure(figsize=(8, 7)) # Set the plot size plt.axis('off') plt.imshow(read_img)</pre>

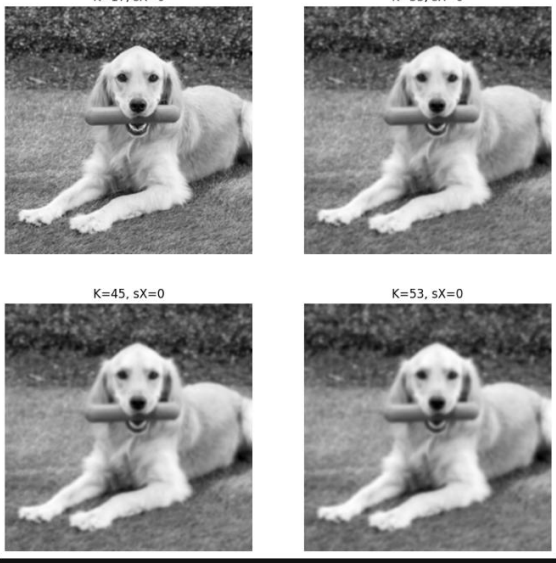


Experiment 3: Blur Image – Average Blur

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># BLUR IMAGE import cv2 import wget import matplotlib.pyplot as plt url_image = wget.download('https://image.chewy.com/is/image/catalog/105549_PT2._AC_SL1500_V1545242865_.jpg') # Image Blur img_data = cv2.imread(url_image) #OpenCV reads the image with BGR color channel img_gray = cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY) # Convert image to Gray (Intensity Image) img_blur1 = cv2.blur(img_gray, ksize = (5, 5)) img_blur2= cv2.blur(img_gray, ksize = (9, 9)) img_blur3 = cv2.blur(img_gray, ksize = (17, 17)) img_blur4 = cv2.blur(img_gray, ksize = (33, 33)) # Display multiple images using subplots list_img = [img_blur1, img_blur2, img_blur3, img_blur4] titl_img = ["K=(5, 5)", "K=(9, 9)", "K=(17, 17)", "K=(33, 33)"] fig, ax = plt.subplots(nrows=2, ncols=2,figsize=(10, 10)) idx = 0 for row in ax: for col in row: col.set_title(titl_img[idx]) col.axis('off') col.imshow(list_img[idx], cmap='gray') idx = idx + 1</pre>

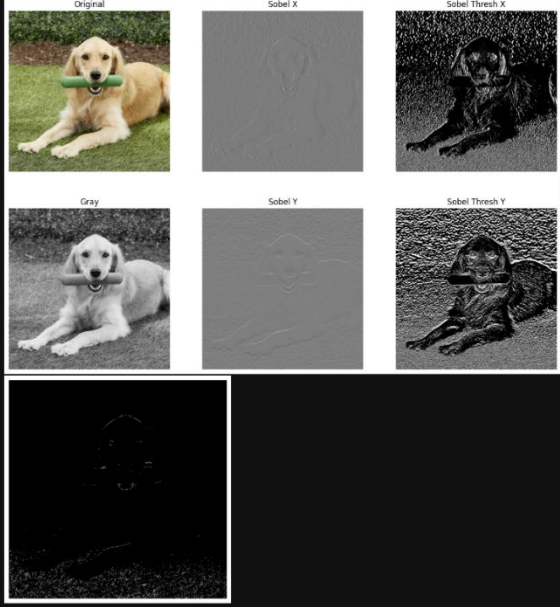


Experiment 4: Blur Image – Gaussian Blur

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># GAUSSIAN IMAGE import cv2 import matplotlib.pyplot as plt url_image = wget.download('https://image.chewy.com/is/image/catalog/105549_PT2._AC_SL1500_V1545242865_.jpg') # Image Blur img_data = cv2.imread(url_image) #OpenCV reads the image with BGR color channel img_gray = cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY) # Convert image to Gray (Intensity Image) # Apply Gaussian Blur img_blur1 = cv2.GaussianBlur(img_gray, ksize = (17, 17), sigmaX=0) img_blur2 = cv2.GaussianBlur(img_gray, ksize = (33, 33), sigmaX=0) img_blur3 = cv2.GaussianBlur(img_gray, ksize = (45, 45), sigmaX=0) img_blur4 = cv2.GaussianBlur(img_gray, ksize = (53, 53), sigmaX=0) # Display multiple images using subplots list_img = [img_blur1, img_blur2, img_blur3, img_blur4] titl_img = ["K=17, sX=0", "K=33, sX=0", "K=45, sX=0", "K=53, sX=0"] fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(10, 10)) idx = 0 for row in ax: for col in row: col.set_title(titl_img[idx]) col.axis('off') col.imshow(list_img[idx], cmap='gray') idx = idx + 1</pre>

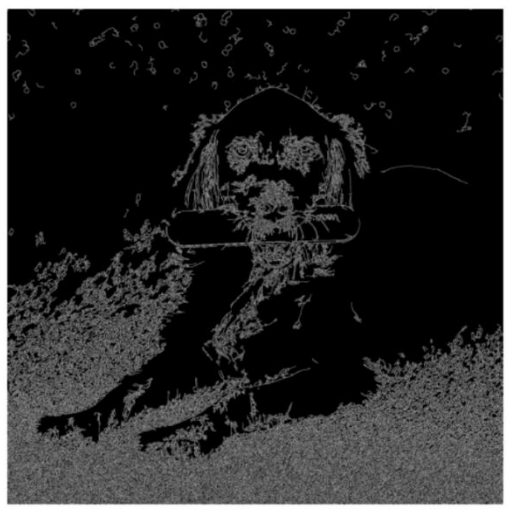


Experiment 5: Edge Detection – Sobel Edge Detection

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre> # SOBEL EDGE DETECTION import numpy as np import cv2 import matplotlib.pyplot as plt # Sobel Edge Detection img_data = cv2.imread(url_image) # OpenCV reads the image with BGR color channel img_gray = cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY) # Convert to gray (intensity image) sobelx = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=5) # x-axis sobely = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=5) # y-axis # Normalize the sobel images img_sobelx = (sobelx + np.abs(np.min(sobelx))) / ((np.max(sobelx)) + np.abs(np.min(sobelx))) * 255.0 img_sobely = (sobely + np.abs(np.min(sobely))) / ((np.max(sobely)) + np.abs(np.min(sobely))) * 255.0 # Thresholding converts the image into binary (0 or 255) ret, thresh_sobelx = cv2.threshold(img_sobelx, 130, 255, cv2.THRESH_BINARY) ret, thresh_sobely = cv2.threshold(img_sobely, 130, 255, cv2.THRESH_BINARY) # Display multiple images using subplots list_img = [img_data, sobelx, thresh_sobelx, img_gray, sobely, thresh_sobely] titl_img = ["Original", "Sobel X", "Sobel Thresh X", "Gray", "Sobel Y", "Sobel Thresh Y"] fig, ax = plt.subplots(nrows=2, ncols=3,figsize=(15, 10)) idx = 0 for row in ax: for col in row: if idx == 0: col.set_title(titl_img[idx]) col.axis('off') col.imshow(cv2.cvtColor(list_img[idx], cv2.COLOR_BGR2RGB)) </pre>

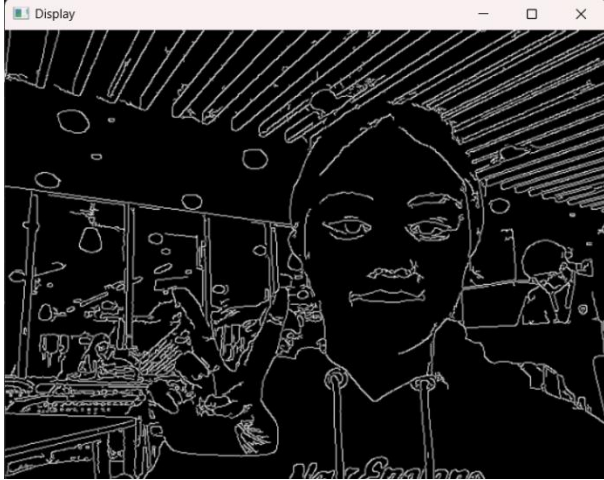
	<pre> else: col.set_title(titl_img[idx]) col.axis('off') col.imshow(list_img[idx], cmap='gray') idx = idx + 1 # Combined x and y derivatives sobel = np.sqrt(np.power(sobelx, 2) + np.power(sobely, 2)) # Normalize the image img_sobel = (sobel + np.abs(np.min(sobel))) / ((np.max(sobel)) + np.abs(np.min(sobel))) * 255.0 # Thresholding converts the image into binary (0 or 255) ret, thresh_sobel = cv2.threshold(img_sobel, 130, 255, cv2.THRESH_BINARY) # Plot the image plt.figure(figsize=(7, 6)) plt.axis('off') plt.imshow(thresh_sobel, cmap='gray') </pre>
--	--

Experiment 6: Edge Detection – Canny Edge Detection

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
<pre>[15]: <matplotlib.image.AxesImage at 0x28ea90b1d10></pre> 	<pre> # CANNY EDGE DETECTION img_data = cv2.imread(url_image) # OpenCV reads the image with BGR color channel #Convert to gray (intensity image) img_gray = cv2.cvtColor(img_data, cv2.COLOR_BGR2GRAY) #Canny Edge Detection img_canny = cv2.Canny(img_gray, threshold1=20, threshold2=200) #Plot the image plt.figure(figsize=(7, 6)) plt.axis('off') plt.imshow(img_canny, cmap='gray') </pre>

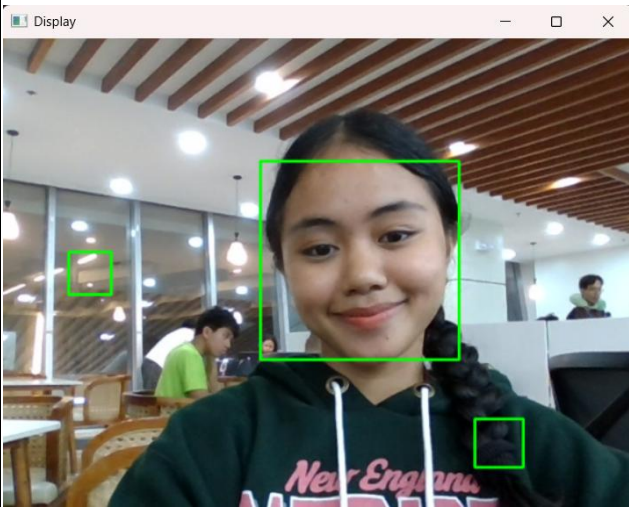


Experiment 7: Real-time Edge Detection

1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># REAL-TIME EDGE DETECTION import cv2 vid = cv2.VideoCapture(0) #Initialize Video Source (Camera) while (True): ret, frame = vid.read() #Get the video frame from the source get_edge = cv2.Canny(frame, threshold1=20, threshold2=200) get_resize = cv2.resize(get_edge, (int(frame.shape[1] * 0.9) , int(frame.shape[0] * 0.9))) #Display the resulting frame cv2.imshow('Display', get_resize) key == cv2.waitKey(20) if key == 27: #PRESS ESC TO QUIT cv2.destroyAllWindows() vid.release() break</pre>



Experiment 8: Real-time Face Detection

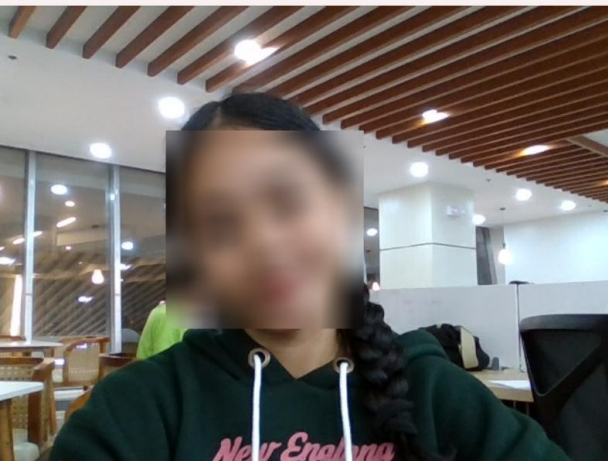
1. Photos / Videos of Physical Test Outputs	2. Photos of Codes
	<pre># REAL-TIME DETECTION import cv2 vid = cv2.VideoCapture(0) # Initialize Video Source (Camera) while (True): ret, frame = vid.read() # Get the video frame from the source #Convert image to grayscale image gray_image = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) #Read the harr_face_detect_classifier.xml harr_cascade = cv2.CascadeClassifier("haarcascade_f rontalface_default.xml") faces = harr_cascade.detectMultiScale(gray_i mage, scaleFactor=1.1, minNeighbors=1) for (x, y, w, h) in faces: #Get the face rectangle coordinates cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255,0), thickness=2) #Draw rectangle get_resize = cv2.resize(frame, (int(frame.shape[1] * 0.9), int(frame.shape[0] * 0.9))) #Display the resulting frame cv2.imshow('Display', get_resize) key = cv2.waitKey(20) if key == 27: # PRESS ESC TO QUIT cv2.destroyAllWindows() vid.release() break</pre>



B. Coding Exercise:

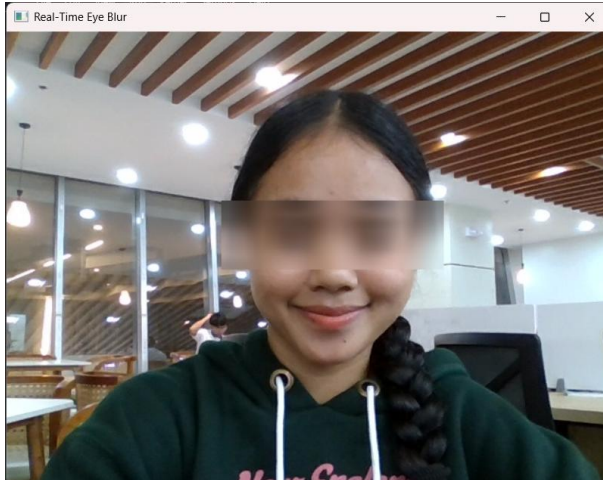
(Please include the following: **1.** Images of test outputs from the Python Notebook, **2.** Photos of the codes that were inputted in the Python Notebook, and **3.** The raw source code that you made for the coding exercise.)

Coding Exercise 1: Real-time Face / Eyes Blur

1. Photos / Videos of Test Outputs	2. Photos of Codes
REAL-TIME FACE BLUR 	<pre># CODING EXERCISE - FACE DETECTION import cv2 # Start video capture vid = cv2.VideoCapture(0) # Load Haar cascade for face detection face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml') while True: ret, frame = vid.read() if not ret: break # Convert to grayscale for detection gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) # Detect faces faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5) for (x, y, w, h) in faces: # Extract the face region face_roi = frame[y:y+h, x:x+w] # Apply Gaussian blur to the face region face_blur = cv2.GaussianBlur(face_roi, (35, 35), 30) # Replace the original face region with the blurred one frame[y:y+h, x:x+w] = face_blur # Show the frame with blurred face(s) cv2.imshow('Real-Time Face Blur', frame) # Press ESC to exit if cv2.waitKey(20) == 27: break # Cleanup vid.release() cv2.destroyAllWindows()</pre>



REAL-TIME EYES BLUR



```
# CODING EXERCISE - EYE DETECTION

import cv2

vid = cv2.VideoCapture(0)

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_eye.xml")

while True:
    ret, frame = vid.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray,
                                          scaleFactor=1.1,
                                          minNeighbors=5,
                                          minSize=(80, 80))

    for (x, y, w, h) in faces:
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = frame[y:y+h, x:x+w]

        # -- detect eyes inside this face
        eyes = eye_cascade.detectMultiScale(roi_gray,
                                             scaleFactor=1.1,
                                             minNeighbors=10,
                                             minSize=(25, 25))

        if len(eyes) >= 1:
            # bounding rectangle that covers "all" detected eyes
            x_vals = [ex for (ex, ey, ew, eh) in eyes] + \
                    [ex + ew for (ex, ey, ew, eh) in eyes]
            y_vals = [ey for (ex, ey, ew, eh) in eyes] + \
                    [ey + eh for (ex, ey, ew, eh) in eyes]
            x1, x2 = min(x_vals), max(x_vals)
            y1, y2 = min(y_vals), max(y_vals)

            # add a little padding so the rectangle looks natural
            pad_x = int((x2 - x1) * 0.6)
            pad_y = int((y2 - y1) * 0.12)
            x1 = max(0, x1 - pad_x)
            x2 = min(w, x2 + pad_x)
            y1 = max(0, y1 - pad_y)
            y2 = min(h, y2 + pad_y)

        else:
            # fallback: rough eye band (25-45 % face height)
            x1 = int(w * 0.20)
            x2 = int(w * 0.80)
            y1 = int(h * 0.25)
            y2 = int(h * 0.45)

        # ---- blur that ONE rectangular ROI ----
        eye_band = roi_color[y1:y2, x1:x2]
        roi_color[y1:y2, x1:x2] = cv2.GaussianBlur(eye_band, (35, 35), 30)

    cv2.imshow("Real-Time Eye Blur", frame)
    if cv2.waitKey(20) & 0xFF == 27: # ESC to quit
        break

vid.release()
```



Raw Source Code:

```
# CODING EXERCISE - FACE DETECTION

import cv2

# Start video capture
vid = cv2.VideoCapture(0)

# Load Haar cascade for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

while True:
    ret, frame = vid.read()
    if not ret:
        break

    # Convert to grayscale for detection
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    for (x, y, w, h) in faces:
        # Extract the face region
        face_roi = frame[y:y+h, x:x+w]

        # Apply Gaussian blur to the face region
        face_blur = cv2.GaussianBlur(face_roi, (35, 35), 30)

        # Replace the original face region with the blurred one
        frame[y:y+h, x:x+w] = face_blur

    # Show the frame with blurred face(s)
    cv2.imshow('Real-Time Face Blur', frame)

    # Press ESC to exit
    if cv2.waitKey(20) == 27:
        break

# Cleanup
vid.release()
cv2.destroyAllWindows()

# CODING EXERCISE - EYE DETECTION

import cv2

vid = cv2.VideoCapture(0)

face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml")
eye_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_eye.xml")

while True:
    ret, frame = vid.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray,
```



```

                                                                    scaleFactor=1.1,
                                                                    minNeighbors=5,
                                                                    minSize=(80, 80))

for (x, y, w, h) in faces:
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = frame[y:y+h, x:x+w]

    # -- detect eyes inside this face
    eyes = eye_cascade.detectMultiScale(roi_gray,
                                        scaleFactor=1.1,
                                        minNeighbors=10,
                                        minSize=(25, 25))

    if len(eyes) >= 1:
        # bounding rectangle that covers *all* detected eyes
        x_vals = [ex for (ex, ey, ew, eh) in eyes] + \
                  [ex + ew for (ex, ey, ew, eh) in eyes]
        y_vals = [ey for (ex, ey, ew, eh) in eyes] + \
                  [ey + eh for (ex, ey, ew, eh) in eyes]
        x1, x2 = min(x_vals), max(x_vals)
        y1, y2 = min(y_vals), max(y_vals)

        # add a little padding so the rectangle looks natural
        pad_x = int((x2 - x1) * 0.6)
        pad_y = int((y2 - y1) * 0.12)
        x1 = max(0, x1 - pad_x)
        x2 = min(w, x2 + pad_x)
        y1 = max(0, y1 - pad_y)
        y2 = min(h, y2 + pad_y)

    else:
        # fallback: rough eye band (25-45% face height)
        x1 = int(w * 0.20)
        x2 = int(w * 0.80)
        y1 = int(h * 0.25)
        y2 = int(h * 0.45)

    # ---- blur that ONE rectangular ROI ----
    eye_band = roi_color[y1:y2, x1:x2]
    roi_color[y1:y2, x1:x2] = cv2.GaussianBlur(eye_band, (35, 35), 30)

cv2.imshow("Real-Time Eye Blur", frame)
if cv2.waitKey(20) & 0xFF == 27: # ESC to quit
    break

vid.release()
cv2.destroyAllWindows()
```



C. Findings ,Observations, and Comments:

Guiding Questions :

1. *What did you learn in the guided experiments? Did you learn something new?*
2. *What are your findings, observations, and comments in the guided experiments?*
3. *Did you solve the coding exercise?*
4. *Are there any difficulties during coding or set-up that you encountered?*
5. *What are your findings, observations, and comments in the coding exercise?*

I successfully displayed the given image using OpenCV and Matplotlib and applied the different types of blurring, which are the average and Gaussian Blur. I also applied the Sobel Edge Detection and combined the x and y gradients and, I made use of the Canny method to detect the edges from the picture taken by a webcam. For real-time face and eye detection, I used the Haar cascades just as what was provided in the laboratory materials and to apply eye blur effect in real-time based for the Coding Exercise. The problems I encountered was I had a difficulty in accessing my webcam and as well as figuring out the first task which I just realized that the image will only be available within 5 seconds in a separate tab. The webcam sometimes fails to detect faces, especially for blurring the face and the eyes, maybe because of the lighting and its minimal error or incompatibilities. Furthermore, converting to grayscale is essential before edge or face detection and increasing the kernel size makes the blur stronger but slower. Gaussian blur produces a more natural blur than simple averaging and Edge detection methods like Sobel and Canny were effective but sensitive to noise. All in all, despite the codes were only being given to us to just copy and type it manually in Jupyter Notebook, I still understood the concept of each task and was amazed by the results from the codes given. It was my first time encountering this kind of coding exercise and I really had fun figuring it all out.